DAY 8

```
#include <iostream>
using namespace std;

// Node structure
class Node {
public:
    int data;
    Node* next;

    Node(int val) {
        data = val;
        next = NULL;
    }
};

// Display linked list
void display(Node* head) {
    Node* temp = head;

    while (temp != NULL) {
        cout << temp->data << " -> ";
        temp = temp->next;
    }

    cout << "NULL" << endl;
}

// 1. INSERT AT HEAD
void insertHead(Node* &head, int val) {
    Node* newNode = new Node(val);

    newNode->next = head;
    head = newNode;
}

// 2. INSERT AT TAIL
void insertTail(Node* &head, int val) {
    Node* newNode = new Node(val);

    if (head == NULL) {
        head = newNode;
        return;
    }
```

```
    Node* temp = head;

    while (temp->next != NULL) {
        temp = temp->next;
    }

    temp->next = newNode;
}

// 3. INSERT AT POSITION (0-based)
void insertAtPosition(Node* &head, int pos, int val) {

    if (pos < 0) return;

    if (pos == 0) {
        insertHead(head, val);
        return;
    }

    Node* temp = head;

    for (int i = 0; i < pos - 1 && temp != NULL; i++) {
        temp = temp->next;
    }

    if (temp == NULL) return;

    Node* newNode = new Node(val);

    newNode->next = temp->next;
    temp->next = newNode;
}

// 4. DELETE FROM HEAD
void deleteHead(Node* &head) {

    if (head == NULL) return;

    Node* temp = head;
    head = head->next;

    delete temp;
}

// 5. DELETE FROM TAIL
void deleteTail(Node* &head) {

    if (head == NULL) return;
```

```
    if (head->next == NULL) {
        delete head;
        head = NULL;
        return;
    }

    Node* temp = head;

    while (temp->next->next != NULL) {
        temp = temp->next;
    }

    delete temp->next;
    temp->next = NULL;
}

// 6. DELETE AT POSITION (0-based)
void deleteAtPosition(Node* &head, int pos) {

    if (head == NULL || pos < 0) return;

    if (pos == 0) {
        deleteHead(head);
        return;
    }

    Node* temp = head;

    for (int i = 0; i < pos - 1 && temp != NULL; i++) {
        temp = temp->next;
    }

    if (temp == NULL || temp->next == NULL) return;

    Node* delNode = temp->next;

    temp->next = delNode->next;

    delete delNode;
}

// 7. LENGTH OF LOOP
int loopLength(Node* head) {

    Node* slow = head;
    Node* fast = head;
```

```
    while (fast != NULL && fast->next != NULL) {

        slow = slow->next;
        fast = fast->next->next;

        if (slow == fast) {

            int count = 1;
            Node* temp = slow->next;

            while (temp != slow) {
                count++;
                temp = temp->next;
            }

            return count;
        }
    }

    return 0;
}


// MAIN FUNCTION
int main() {

    Node* head = NULL;

    // Insertion
    insertHead(head, 20);
    insertHead(head, 10);
    insertTail(head, 40);
    insertTail(head, 50);
    insertAtPosition(head, 2, 30);

    cout << "Original Linked List: ";
    display(head);

    // Deletion
    deleteHead(head);

    cout << "After deleting head: ";
    display(head);

    deleteTail(head);

    cout << "After deleting tail: ";
    display(head);
```

```
    deleteAtPosition(head, 1);

    cout << "After deleting position 1: ";
    display(head);

    // Kth node from end
    int k = 2;

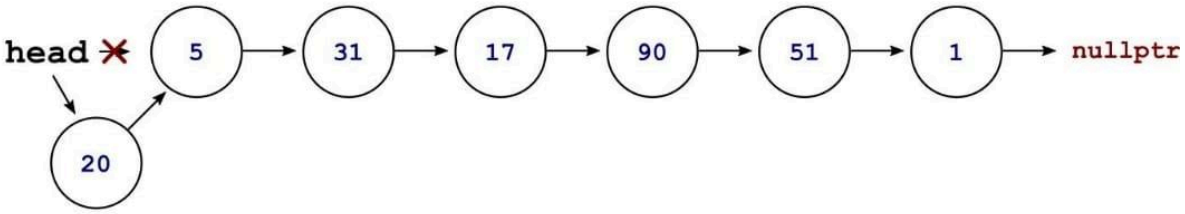
    Node* ans = kthFromEnd(head, k);

    if (ans != NULL)
        cout << "Kth node from end: " << ans->data << endl;
    else
        cout << "Invalid K" << endl;

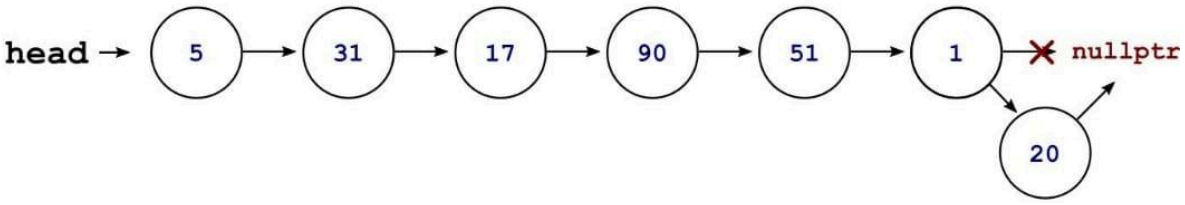
    // Loop length
    cout << "Length of loop: " << loopLength(head) << endl;

    return 0;
}
```

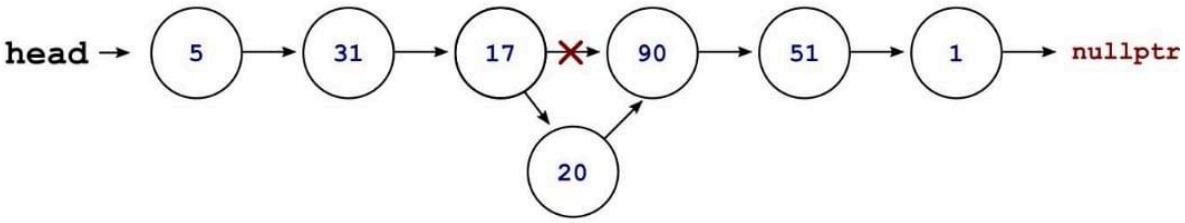
Insert at front (push front)

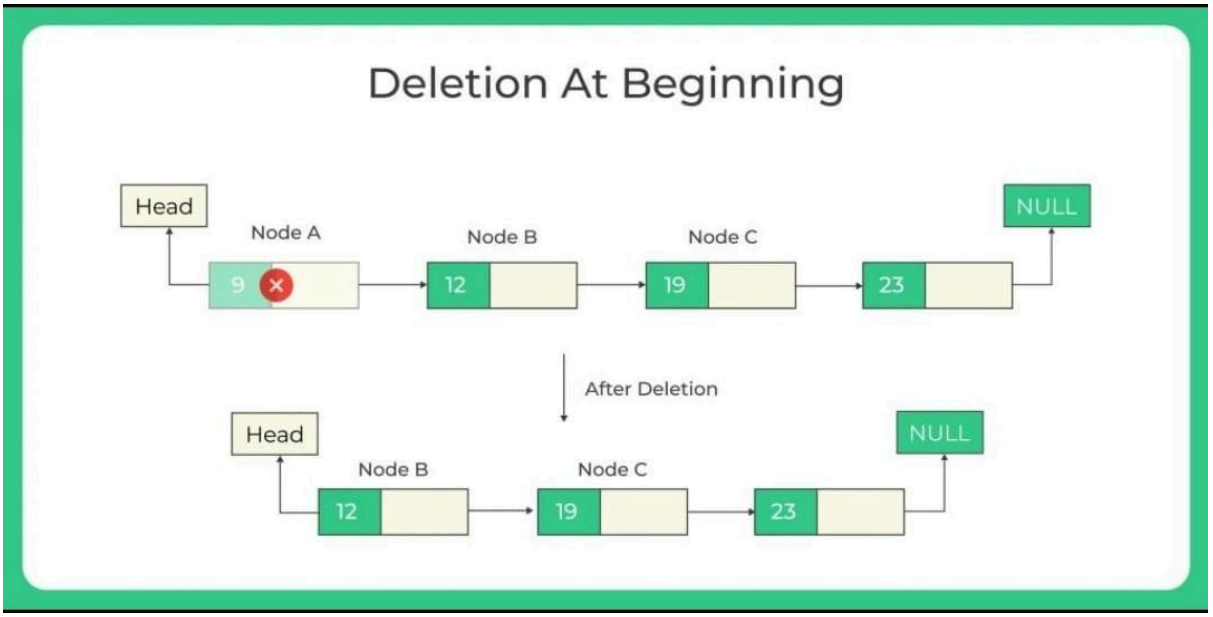


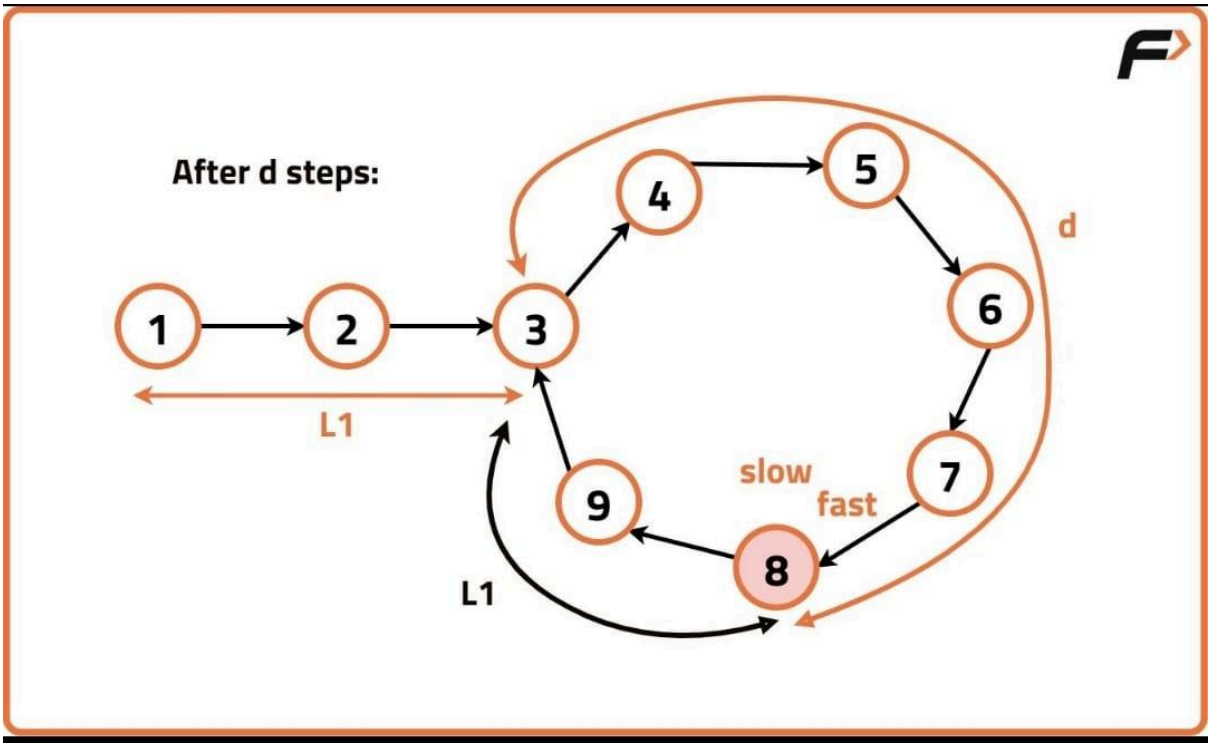
Insert at back (append)



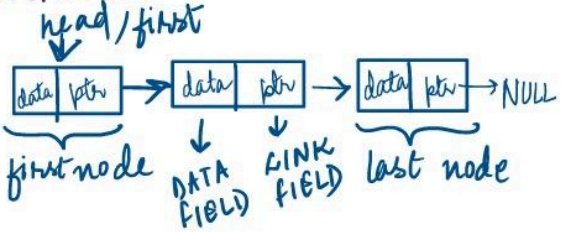
Insert into middle




Deletion At Beginning
Head
Node A
Node B
Node C
NULL
9
12
19
23
After Deletion
Head
Node B
Node C
NULL
12
19
23


After d steps:
1
2
3
4
5
6
7
8
9
d
L1
L1
slow
fast

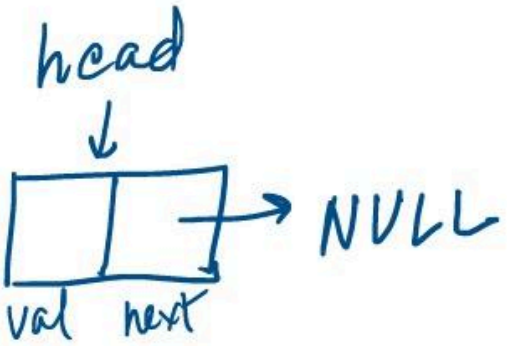
Linked List → collection of nodes

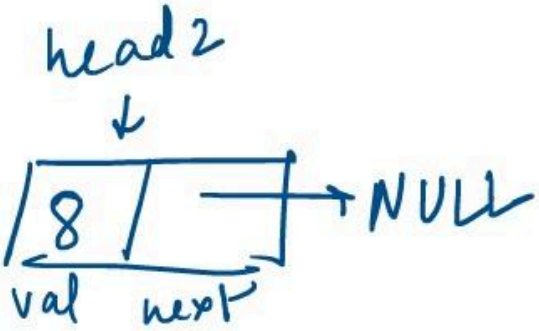


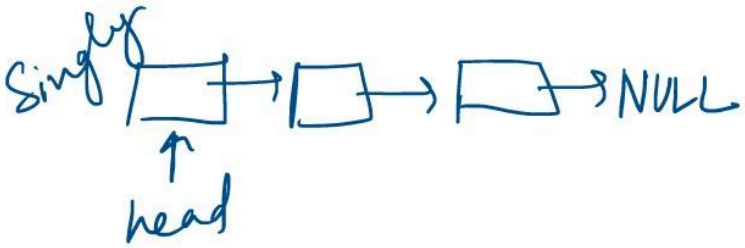
* Struct LLNode
{ int data;
LLNode * next;
}

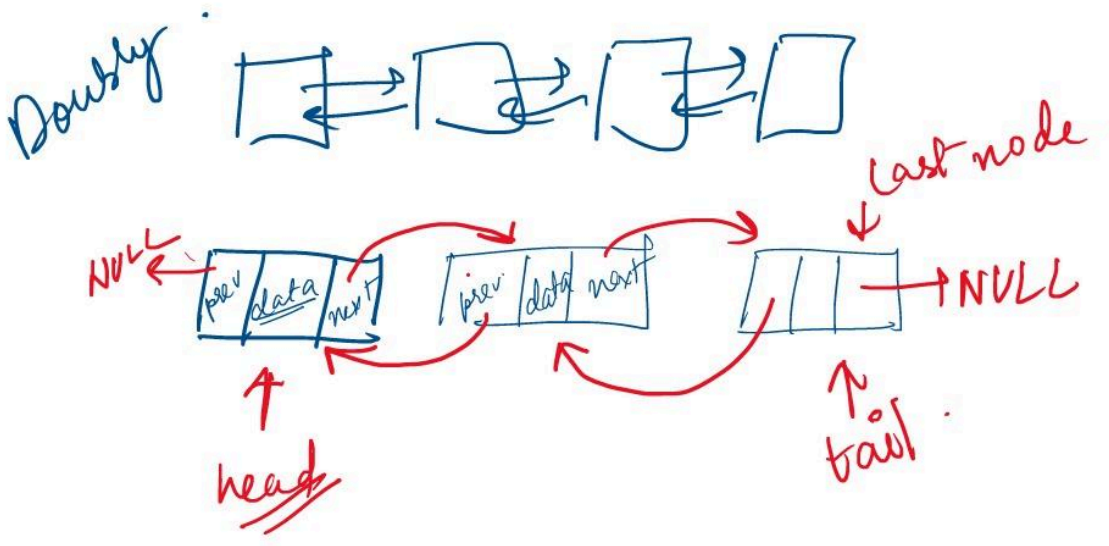
Class node
{ public:
int val;
node * next;
}

* To create a new node:
node * p = new node;
* To add values to a node:
p → val = 10;


head
NULL
val
next


head2
8
NULL
val
next


Singly
NULL
head


Doubly
last node
NULL
prev
data
next
NULL
head
tail

LEETCODE

141. Linked List Cycle

Solved

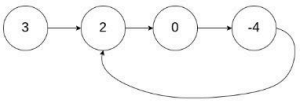
Easy | Topics | Companies

Given `head`, the head of a linked list, determine if the linked list has a cycle in it.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the `next` pointer. Internally, `pos` is used to denote the index of the node that tail's `next` pointer is connected to. **Note that `pos` is not passed as a parameter**.

Return `true` *if there is a cycle in the linked list*. Otherwise, return `false`.

Example 1:




```
Input: head = [3,2,0,-4], pos = 1
Output: true
Explanation: There is a cycle in the linked list, where the tail connects to the 1st node (0-indexed).
```

Example 2:

C++ | Auto

```cpp
*/
class Solution {
public:
    bool hasCycle(ListNode *head) {
        if(!head) return false;
        ListNode* fast=head;
        ListNode* slow=head;

        while(fast->next && fast->next->next){
            slow=slow->next;
            fast=fast->next->next;
            if(slow==fast) return true;
        }
        return false;
    }
};
// // slow,fast-3
// slow-2,fast-0
// slow-0,fast-2
```

Saved

Testcase | Test Result

Accepted Runtime: 1 ms

Case 1 | Case 2 | Case 3

Problem List | Submit

Description | Editorial | Solutions | Submissions

876. Middle of the Linked List

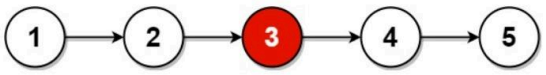
Solved

Easy | Topics | Companies

Given the `head` of a singly linked list, return *the middle node of the linked list*.

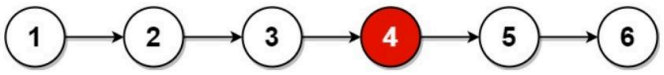
If there are two middle nodes, return **the second middle** node.

Example 1:




```
Input: head = [1,2,3,4,5]
Output: [3,4,5]
Explanation: The middle node of the list is node 3.
```

Example 2:




```
Input: head = [1,2,3,4,5,6]
Output: [4,5,6]
Explanation: Since the list has two middle nodes with values 3 and 4, we return the second one
```

Code | Accepted

C++ | Auto

```cpp
class Solution {
public:
    ListNode* middleNode(ListNode* head) {
        int count=0;
        ListNode* temp=head;

        while(temp){
            temp=temp->next;
            count++;
        }

        ListNode* slow=head,*fast=head;
        while(fast->next && fast->next->next){
            slow=slow->next;
            fast=fast->next->next;
            if(slow==fast) break;
        }
        return count%2==0?slow->next:slow;
    }
```

Saved

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input